

Supervised Learning: Techniques and Implementation

Dataset Description

The Iris dataset, utilized in this project, is a renowned dataset introduced by R.A. Fisher in 1936. It contains measurements of 150 iris flowers, distributed evenly across three species: Iris-setosa[0-49], Iris-versicolor[50-99], and Iris-virginica[100-149]. Each entry includes the following attributes:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width
5. Class (Species)

The dataset was sourced from the [UCI Machine Learning Repository](#).

Libraries and Tools Used:

NumPy

- Efficiently handles numerical computations and array manipulations.

Pandas

- Facilitates reading, writing, and manipulation of data in tabular format.

Matplotlib

- A visualization library built on NumPy arrays.
- Provides various plots such as line, bar, scatter, and histogram.

Pyplot:

- Pyplot makes Matplotlib work like MATLAB.

Seaborn

- Built on top of Matplotlib, it enhances statistical plotting.
- Provides beautiful default styles and colour palettes to make plots attractive.

Scikit-learn

- Provides tools for machine learning, preprocessing, and model evaluation. Key modules used include:
 - **train_test_split** for dataset splitting.
 - **StandardScaler** for feature normalization.
 - Classifiers like **GaussianNB**, **SVC**, and **KNeighborsClassifier**.
 - Metrics such as **accuracy_score**, **classification_report**, and **confusion_matrix**.
-

Definitions:

iloc () function:

- Stands for "integer location".
- Used for accessing and retrieving data from Pandas DataFrame objects using integer-based indexing.

Train-Test Split:

- A method to divide the dataset into training and testing subsets to evaluate model performance.

StandardScaler:

- Normalizes data by estimating mean and standard deviation on the training set and applying them to scale the dataset.

scaler.fit():

- Similar to fitting a model, we can fit our scaling to the data using the fit method on the training set. This simply estimates the mean and standard deviation.

Scaler.transform():

- To apply the scaling, we can use the transform method, which will subtract the mean and divide by the standard deviation.

GaussianNB:

- A Naive Bayes classifier for continuous data that assumes features follow a Gaussian distribution.
- "Naive" is used because we are using the features which are independent of each other.

SVC (Support Vector Classifier):

- Specific implementation of the Support Vector Machine algorithm that is designed specifically for classification tasks.
- Finds the optimal hyperplane to classify data points for classification tasks.

Kernel

- Kerneling is a trick used in machine learning (especially in **Support Vector Machines** or SVM) to **solve problems** where the **data is not easily separable**. Imagine you have data points on a piece of paper, and you need to draw a line to separate them into two groups.
- **Linear Kernel:**
 - Formula: $K(x, x') = x^T x' K(x, x') = x^T x'$
 - This is the simplest kernel, used when the data is linearly separable. It does not transform the data into a higher-dimensional space.
- **Polynomial Kernel:**
 - Formula: $K(x, x') = (x^T x' + c)^d K(x, x') = (x^T x' + c)^d$
 - This kernel maps the data into a polynomial feature space. The degree d controls the flexibility of the decision boundary.
- **Radial Basis Function (RBF) Kernel (Gaussian Kernel):**
 - Formula: $K(x, x') = \exp\left[-\gamma \|x - x'\|^2\right] K(x, x') = \exp(-\gamma \|x - x'\|^2)$
 - This is one of the most commonly used kernels. It maps the data into an infinite-dimensional space, allowing for very complex decision boundaries. The parameter γ controls the influence of individual training examples.
- **Sigmoid Kernel:**
 - Formula: $K(x, x') = \tanh(\alpha x^T x' + c) K(x, x') = \tanh(\alpha x^T x' + c)$
 - This kernel is similar to the activation function used in neural networks. It can be used in certain cases but is less common than the RBF or polynomial kernels.

KNeighborsClassifier:

- Implements the k-nearest neighbors algorithm to classify data based on nearest neighbors.
- **Accuracy peaks at K = 7 to 10** because it balances sensitivity and generality, reducing noise and avoiding overfitting or underfitting.
- If **K = 1, 2, 3**: the model becomes too sensitive to train data and thus may include noise and outliers which further **leads to Overfitting Condition**. Hence the accuracy reduces
- If **K=15, 16, 17**: the model becomes too general to train the data and thus it smooths out the noise and outliers which further **leads to Underfitting Condition**. Hence accuracy again reduces.

classifier.fit():

- The learning algorithm / estimator instance is first fitted to the model, i.e., it must *learn* from the model. This is done by passing our training set to the fit method.

classifier.predict():

- The predict() will perform a prediction for each test instance, and it usually accepts only a single input.

confusion_matrix():

- Summarizes prediction results in matrix form, showing correct and incorrect predictions per class.

seaborn.heatmap() / sns.heatmap():

- Plot rectangular data as a color-encoded matrix.

Parameters:

(i) **data**: Here, the data input is the 'result'.

(ii) **annot**: bool or rectangular dataset, optional.

If True, write the data value in each cell. If an array-like object has the same shape as data, then use this to annotate the heatmap instead of the data. Note that DataFrames will match on position, not index.

In this example, in each confusion matrix, correctly and incorrectly classified test sample

numbers are shown.

(iii) **fmt**: string formatting code to use when adding annotations.

G or g (General Format) suffix in Python: It can be interpreted as the total number of digits before the decimal; otherwise, it represents the total number of zeros after the decimal point but before the significant digits.

In this example, `fmt = 'g'` is used.

(iv) **xticklabels, yticklabels**: Here, sequentially, three classes, 'Setosa', 'Versicolor', and 'Virginica' are marked.

classification_report: Build a text report showing the main classification metrics.

(i) **Precision**: $TP / (TP + FP)$

(ii) **Recall / Sensitivity**: $TP / (TP + FN)$

(iii) **F1 Score** = $(2 * Precision * Recall) / (Precision + Recall)$

True Positive (TP): A true positive is an outcome where the model *correctly* predicts the *positive* class.

True Negative (TN): A true negative is an outcome where the model *correctly* predicts the *negative* class.

False Positive (FP): A false positive is an outcome where the model *incorrectly* predicts the *positive* class.

False Negative (FN): A false negative is an outcome where the model *incorrectly* predicts the *negative* class.

Support: Support is a measure of the number of times an item set appears in a dataset. Here, the item set is a test data sample.

Macro Average: Average the precision, recall, and F1 scores across all classes to get the final macro-averaged precision, recall, and F1 scores, respectively. You may do the hand calculations and cross-check the output you get.

Weighted Average:

Weighted Average of Precision =
$$\frac{\text{Sum (Individual Class Precision * Individual Class Support)}}{\text{Total Support.}}$$

In a similar way, you can do it for the rest of the recall and F1 score. You may do the hand calculations and cross-check the output you get.

accuracy_score: In Python, the `accuracy_score` function of the `sklearn.metrics` package calculates the accuracy score for a set of predicted labels against the true labels.

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

The IRIS data used in the example has a total of 150 data samples that comprise 50 from each class of IRIS Setosa, IRIS Virginica, and IRIS Versicolor.

During the training and test split, 75% was used for training and 25% for testing purposes. 25% of 150 is 38.

That is the reason the total support count is 38.

For instance, depending on the output, among the 38, there can be 16 Iris Setosa, 14 Iris Virginica, and 8 Iris Versicolor. The same number will reflect during the prediction evaluations in the confusion matrix.

Preprocessing Steps

Data Loading:

```
path = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
headernames = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']
data = pd.read_csv(path, names=headernames)
```

Feature Extraction:

```
X = data.iloc[:, :-1].values # Features
y = data.iloc[:, 4].values # Labels
```

Data Splitting:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
```

Normalization:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Implemented Algorithms:

Naive Bayes Classifier

- A probabilistic algorithm based on Bayes' theorem.

Implementation:

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
```

Evaluation:

```
y_pred = classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Support Vector Machines (SVM)

- Identifies the optimal hyperplane to classify data points.

Kernel Comparison:

```
from sklearn.svm import SVC
accuracy_list = []
for kernel in ['linear', 'poly', 'rbf', 'sigmoid']:
    clf = SVC(kernel=kernel)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracy_list.append(accuracy)
print("Accuracy of {kernel} kernel: {accuracy}")
```

Visualization:

```
plt.figure(figsize=(8, 6))
sns.barplot(x=['Linear', 'Polynomial', 'RBF', 'Sigmoid'], y=accuracy_list)
plt.xlabel('Kernel')
plt.ylabel('Accuracy')
plt.title('Accuracy of Different SVM Kernels')
plt.ylim(0.8, 1.0)
plt.show()
```

K-Nearest Neighbors (KNN)

- Classifies data based on the majority class among the nearest neighbors.

Implementation:

```
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=9)
classifier.fit(X_train, y_train)
```

Evaluation:

```
y_pred = classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Model Evaluation:**Confusion Matrix**

```
from sklearn.metrics import confusion_matrix
result = confusion_matrix(y_test, y_pred)
sns.heatmap(result, annot=True, fmt='g',
             xticklabels=['Setosa', 'Versicolor', 'Virginica'],
             yticklabels=['Setosa', 'Versicolor', 'Virginica'])
plt.ylabel('Prediction')
plt.xlabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```